Java Scanner: Reading Files

Study notes • methods, loops, execution traces, and examples

Core idea: a `Scanner` reads from a source such as a file. `hasNext...` methods check upcoming input without consuming it; `next...` methods read and consume input.

1. The six methods at a glance

Method	What it checks or reads	Consumes input?
`hasNext()`	Checks whether another whitespace-separated token exists, regardless of type.	No
`next()`	Reads the next token and returns it as a String.	Yes
`hasNextInt()`	Checks whether the next token can be interpreted as a Java int.	No
`nextInt()`	Reads the next token as an int.	Yes
`hasNextLine()`	Checks whether another line is available.	No
`nextLine()`	Reads the remaining content of the current line as a String.	Yes, through the line ending

Remember: the check method does not consume the token/line. The matching next method does. A loop repeats the check and reading operation until its condition becomes false.

2. Example A — hasNextInt() + nextInt()

File `data.txt` contains:

```
10 20 30
40 50
```

Code:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class Main {
    public static void main(String[] args)
            throws FileNotFoundException {

        Scanner sc = new Scanner(new File("data.txt"));

        while (sc.hasNextInt()) {
            int number = sc.nextInt();
            System.out.println(number);
        }

        sc.close();
    }
}
```

Output:

```
10
20
30
40
50
```

Loop trace (line breaks count as whitespace for token scanning):

Check #	hasNextInt()	Body action	Remaining tokens after body
1	true (next = 10)	nextInt() reads 10; prints 10	20 30 40 50
2	true (next = 20)	nextInt() reads 20; prints 20	30 40 50
3	true (next = 30)	nextInt() reads 30; prints 30	40 50
4	true (next = 40)	nextInt() reads 40; prints 40	50
5	true (next = 50)	nextInt() reads 50; prints 50	none
6	false	Body does not execute; loop ends	none

The condition is checked six times, but the loop body runs five times.

3. Example B — hasNextLine() + nextLine()

File `data.txt` contains:

```
Alice is 20 years old
Bob is 21 years old
Charlie is 22 years old
```

Code:

```
Scanner sc = new Scanner(new File("data.txt"));

while (sc.hasNextLine()) {
    String line = sc.nextLine();
    System.out.println(line);
}

sc.close();
```

Output:

```
Alice is 20 years old
Bob is 21 years old
Charlie is 22 years old
```

Check #	hasNextLine()	nextLine() returns	Result
1	true	Alice is 20 years old	Print first line
2	true	Bob is 21 years old	Print second line
3	true	Charlie is 22 years old	Print third line
4	false	Not called	Loop ends

Important: `nextLine()` returns a whole line, including spaces between words. It does not split the line into individual tokens.

4. Example C — nextInt() versus nextLine()

Suppose the file contains one line:

```
25 30 40
```

A. Read tokens as integers

```
Scanner sc = new Scanner(new File("data.txt"));

int a = sc.nextInt();
int b = sc.nextInt();

System.out.println(a);
System.out.println(b);
```

Output:

```
25
30
```

Operation	Value returned	Remaining tokens
Initial state	—	25 30 40
nextInt()	25	30 40
nextInt()	30	40

B. Read the whole line

```
Scanner sc = new Scanner(new File("data.txt"));

String line = sc.nextLine();
System.out.println(line);
```

Output:

```
25 30 40
```

Here, the entire line is returned as one String, not three separate integers.

5. Example D — hasNext() + next()

File `data.txt` contains:

```
Alice 25 Turin
Bob 21 Milan
```

Code:

```
Scanner sc = new Scanner(new File("data.txt"));

while (sc.hasNext()) {
    String token = sc.next();
    System.out.println(token);
}

sc.close();
```

Output:

```
Alice
25
Turin
Bob
21
Milan
```

Execution trace:

Check #	hasNext()	next() consumes	Remaining tokens
1	true	Alice	25 Turin Bob 21 Milan
2	true	25	Turin Bob 21 Milan
3	true	Turin	Bob 21 Milan
4	true	Bob	21 Milan
5	true	21	Milan
6	true	Milan	none
7	false	Body does not execute	none

`next()` returns each token as a String. It does not care whether the token looks numeric.

6. Example E — hasNext() versus hasNextInt()

File contents:

```
10 hello 20
```

A. hasNext() + next()

```
while (sc.hasNext()) {
    System.out.println(sc.next());
}
```

Output:

```
10
hello
20
```

B. hasNextInt() + nextInt()

```
while (sc.hasNextInt()) {
    System.out.println(sc.nextInt());
}
```

Output:

```
10
```

The integer loop stops at `hello`, because that token cannot be read as an int. The invalid token is not consumed by `hasNextInt()`, so the loop condition stays false if checked again.

7. Nested loops: reading each line, then each integer

For a file whose lines contain integers, you can process the file line by line and then tokenize each line separately. This is useful when line boundaries matter.

File contents:

```
10 20 30
40 50 60
```

Code:

```
Scanner sc = new Scanner(new File("data.txt"));

while (sc.hasNextLine()) {
    String line = sc.nextLine();
    Scanner lineScanner = new Scanner(line);

    while (lineScanner.hasNextInt()) {
        int number = lineScanner.nextInt();
        System.out.println(number);
    }

    lineScanner.close();
}

sc.close();
```

Output:

```
10
20
30
40
50
60
```

Outer loop = one iteration per line. Inner loop = one iteration per integer token on that line. If the file contains only whitespace-separated integers and line boundaries do not matter, a single `while (sc.hasNextInt())` loop is simpler.

8. Exam checklist

If the task asks you to…	Typical pattern
Read every integer token	`while (sc.hasNextInt()) { int n = sc.nextInt(); }`
Read every token as text	`while (sc.hasNext()) { String s = sc.next(); }`
Read every line	`while (sc.hasNextLine()) { String line = sc.nextLine(); }`
Read only the next integer	`if (sc.hasNextInt()) { int n = sc.nextInt(); }`

Common mistake: if a file contains `10 hello 40`, `while (sc.hasNextInt())` reads 10 and then stops at hello. It does not skip the invalid token to find 40. If you need to continue after invalid tokens, you must explicitly consume or handle them.

Technical note: `hasNextInt()` checks the next token, not every token in the file. A Scanner may read ahead internally to identify token boundaries, but it does not validate the whole file at once.